

COS20019 - Cloud Computing Architecture

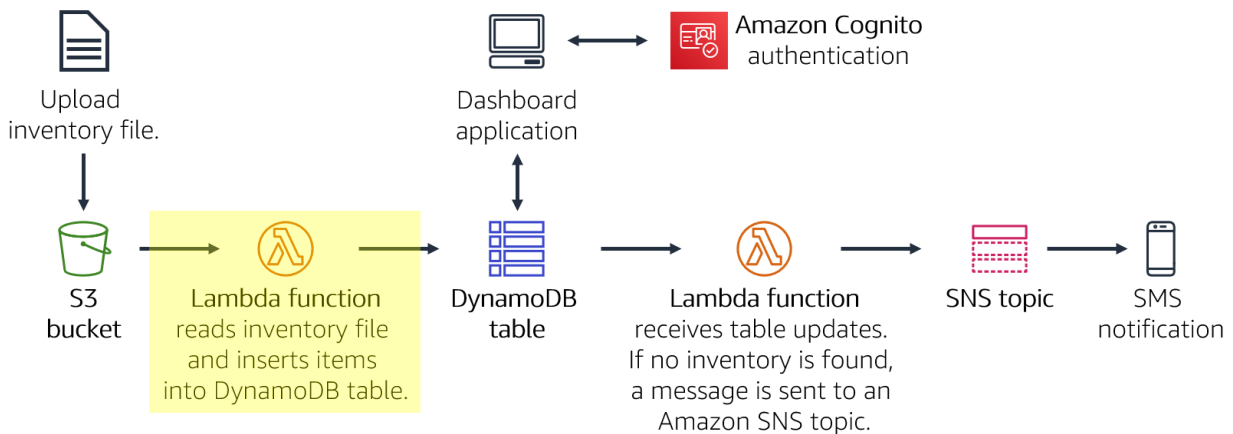
Week 9: ACA Module 13 Guided Lab - Implementing a Serverless Architecture with AWS Lambda

Author: Trac Duc Anh Luong - ID: 103488117

Due Date: 09/11/2023

Task 1: Creating a Lambda function to load data

In this task, you will create a *Lambda function* that will process an inventory file. The Lambda function will read the file and insert information into a DynamoDB table.



5. In the **AWS Management Console**, on the Services menu, choose **Lambda**.
6. Choose Create function

Blueprints are code templates for writing Lambda functions. Blueprints are provided for standard Lambda triggers, such as creating Amazon Alexa skills and processing Amazon Kinesis Data Firehose streams. This lab provides you with a pre-written Lambda function, so you will use the **Author from scratch** option.
7. Configure the following settings:
 - **Function name:** **Load-Inventory**
 - **Runtime:** *Python 3.7*
 - Expand **Choose or create an execution role**.
 - **Execution role:** *Use an existing role*
 - **Existing role:** *Lambda-Load-Inventory-Role*

Create function - Lambda

https://us-east-1.console.aws.amazon.com/lambda/home?region=us-east-1#/create/function

Services Search [Alt+S]

N. Virginia voclabs/user2752985=103488117@student.swin.edu.au @ 1219-6403...

Lambda > Functions > Create function

Create function [Info](#)

AWS Serverless Application Repository applications have moved to [Create application](#).

☒ **Author from scratch**
Start with a simple Hello World example.

☐ **Use a blueprint**
Build a Lambda application from sample code and configuration presets for common use cases.

☐ **Container image**
Select a container image to deploy for your function.

Basic information

Function name
Enter a name that describes the purpose of your function.

Use only letters, numbers, hyphens, or underscores with no spaces.

Runtime [Info](#)
Choose the language to use to write your function. Note that the console code editor supports only Node.js, Python, and Ruby.

Architecture [Info](#)
Choose the instruction set architecture you want for your function code.
☒ x86_64
☐ arm64

Permissions [Info](#)
By default, Lambda will create an execution role with permissions to upload logs to Amazon CloudWatch Logs. You can customize this default role later when adding triggers.

▼ **Change default execution role**

Execution role
Choose a role that defines the permissions of your function. To create a custom role, go to the [IAM console](#).

☐ Create a new role with basic Lambda permissions

☒ Use an existing role

☐ Create a new role from AWS policy templates

Existing role
Choose an existing role that you've created to be used with this Lambda function. The role must have permission to upload logs to Amazon CloudWatch Logs.

[View the Lambda-Load-Inventory-Role role](#) on the IAM console.

► **Advanced settings**

Cancel **Create function**

CloudShell Feedback © 2023, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

This role gives the Lambda function permissions so that it can access Amazon S3 and DynamoDB.

- Choose Create function
- Scroll down to the **Code source** section, and in the **Environment** pane, choose `lambda_function.py`.
- In the code editor, delete all the code.
- In the **Code source** editor, copy and paste the following code:

```
# Load-Inventory Lambda function
#
# This function is triggered by an object being created in an Amazon S3 bucket.
# The file is downloaded and each line is inserted into a DynamoDB table.
import json, urllib, boto3, csv
# Connect to S3 and DynamoDB
s3 = boto3.resource('s3')
dynamodb = boto3.resource('dynamodb')
# Connect to the DynamoDB tables
inventoryTable = dynamodb.Table('Inventory');
# This handler is run every time the Lambda function is triggered
def lambda_handler(event, context):
    # Show the incoming event in the debug log
    print("Event received by Lambda function: " + json.dumps(event, indent=2))
    # Get the bucket and object key from the Event
    bucket = event['Records'][0]['s3']['bucket']['name']
    key = urllib.parse.unquote_plus(event['Records'][0]['s3']['object']['key'])
    localFilename = '/tmp/inventory.txt'
    # Download the file from S3 to the local filesystem
    try:
        s3.meta.client.download_file(bucket, key, localFilename)
    except Exception as e:
        print(e)
        print('Error getting object {} from bucket {}. Make sure they exist and your bucket is in the
        same region as this function.'.format(key, bucket))
        raise e
    # Read the Inventory CSV file
    with open(localFilename) as csvfile:
        reader = csv.DictReader(csvfile, delimiter=',')
        # Read each row in the file
        rowCount = 0
        for row in reader:
            rowCount += 1
            # Show the row in the debug log
            print(row['store'], row['item'], row['count'])
            try:
                # Insert Store, Item and Count into the Inventory table
                inventoryTable.put_item(
                    Item={
                        'Store': row['store'],
                        'Item': row['item'],
                        'Count': int(row['count'])})
            except Exception as e:
                print(e)
```

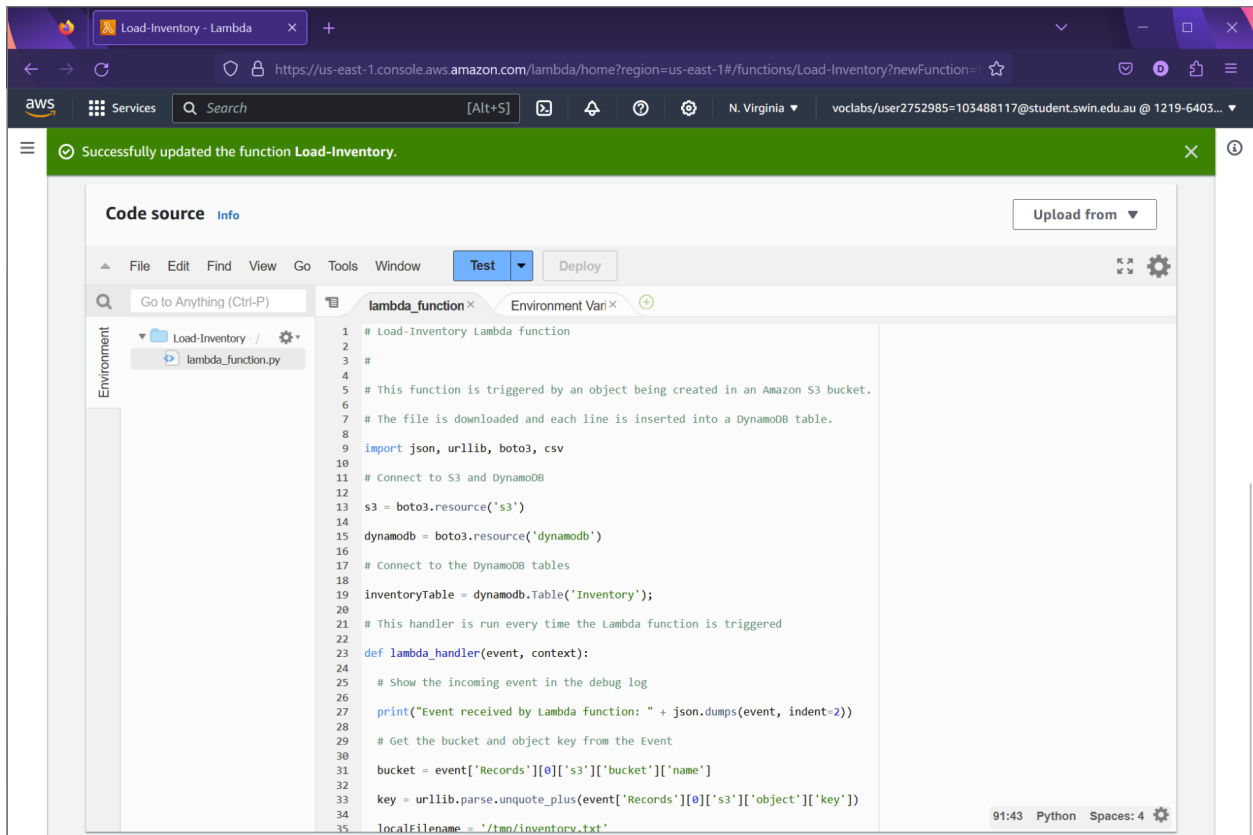
```
print("Unable to insert data into DynamoDB table".format(e))
# Finished!
return "%d counts inserted" % rowCount
```

Examine the code. It performs the following steps:

- Download the file from Amazon S3 that triggered the event
- Loop through each line in the file
- Insert the data into the DynamoDB *Inventory* table

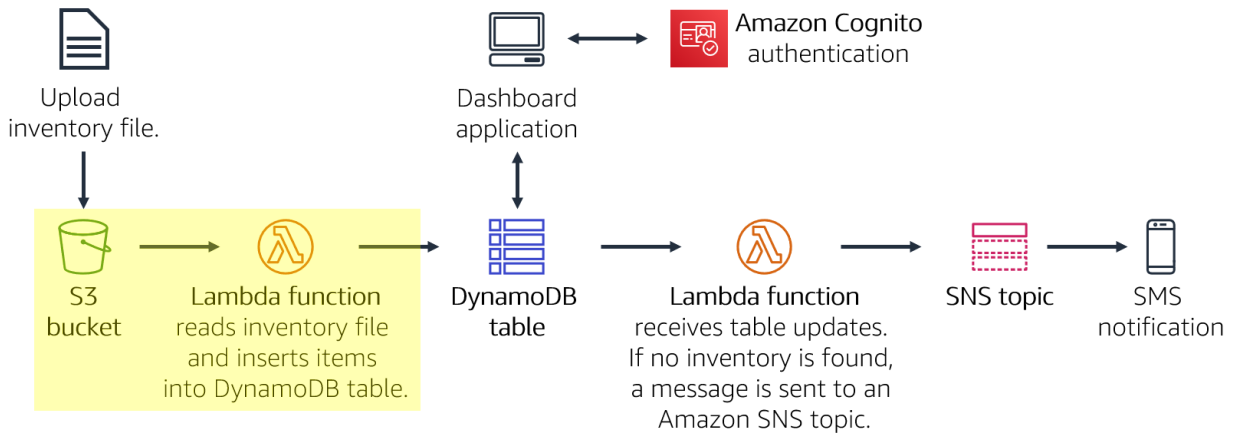
12. Choose Deploy to save your changes.

Next, you will configure Amazon S3 to trigger the Lambda function when a file is uploaded.



Task 2: Configuring an Amazon S3 event

Stores from around the world provide inventory files to load into the inventory tracking system. Instead of uploading their files via FTP, the stores can upload them directly to Amazon S3. They can upload the files through a webpage, a script, or as part of a program. When a file is received, it triggers the Lambda function. This Lambda function will then load the inventory into a DynamoDB table.



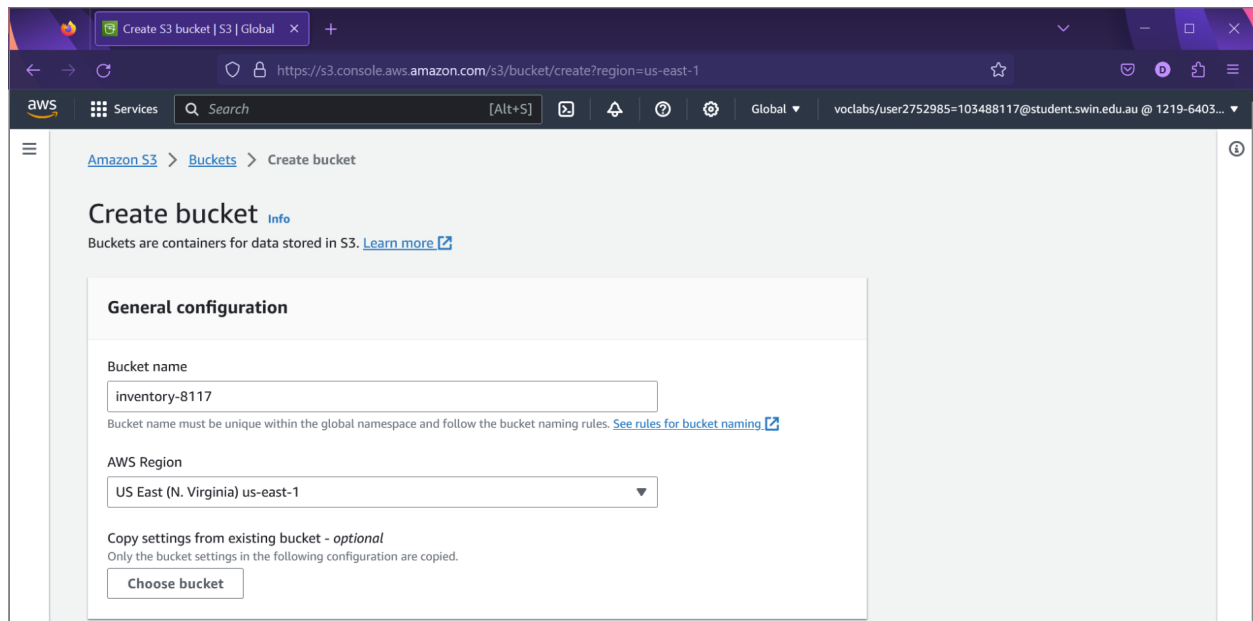
In this task, you will create an S3 bucket and configure it to trigger the Lambda function.

13. On the Services menu, choose **S3**.

14. Choose Create bucket

Each bucket must have a unique name, so you will add a random number to the bucket name. For example: *inventory-123*

15. For **Bucket name** enter: **inventory-<number>** (Replace with a random number)



16. Choose Create bucket

You might receive an error that states: *The requested bucket name is not available*. If you get this error, choose the first **Edit** link, change the bucket name, and try again until the bucket name is accepted.

You will now configure the bucket to automatically trigger the Lambda function when a file is uploaded.

17. Choose the name of your *inventory*- bucket.

18. Choose the **Properties** tab.

19. Scroll down to **Event notifications**.

You will configure an event to trigger when an object is created in the S3 bucket.

20. Click Create event notification then configure these settings:

- **Name:** **Load-Inventory**
- **Event types:** *All object create events*
- **Destination:** *Lambda Function*
- **Lambda function:** *Load-Inventory*
- Choose Save changes

The screenshot shows the AWS S3 console interface for creating an event notification. The breadcrumb trail is: Amazon S3 > Buckets > inventory-8117 > Create event notification. The page title is 'Create event notification' with an 'Info' link. A note states: 'To enable notifications, you must first add a notification configuration that identifies the events you want Amazon S3 to publish and the destinations where you want Amazon S3 to send the notifications.'

General configuration

Event name:
Event name can contain up to 255 characters.

Prefix - optional
Limit the notifications to objects with key starting with specified characters.

Suffix - optional
Limit the notifications to objects with key ending with specified characters.

Event types
Specify at least one event for which you want to receive notifications. For each group, you can choose an event type for all events, or you can choose one or more individual events.

Object creation

☒ **All object create events**
s3:ObjectCreated:*

☐ **Put**
s3:ObjectCreated:Put

☐ **Post**
s3:ObjectCreated:Post

☐ **Copy**
s3:ObjectCreated:Copy

☐ **Multipart upload completed**
s3:ObjectCreated:CompleteMultipartUpload

Create event notification - S3

https://s3.console.aws.amazon.com/s3/bucket/inventory-8117/property/notification/create?region=us-east-1

Services Search [Alt+S] Global voclabs/user2752985=103488117@student.swin.edu.au @ 1219-6403...

Lifecycle

- ☐ Lifecycle transition events
s3:LifecycleTransition
- ☐ All lifecycle expiration events
s3:LifecycleExpiration:*
- ☐ Object not tracked by replication
s3:Replication:OperationNotTracked
- ☐ Object failed to replicate
s3:Replication:OperationFailedReplication
- ☐ Object expired
s3:LifecycleExpiration:Delete
- ☐ Delete marker added by Lifecycle for a versioned object
s3:LifecycleExpiration:DeleteMarkerCreated

Intelligent-Tiering

- ☐ Intelligent-Tiering archive events
s3:IntelligentTiering

Destination

Before Amazon S3 can publish messages to a destination, you must grant the Amazon S3 principal the necessary permissions to call the relevant API to publish messages to an SNS topic, an SQS queue, or a Lambda function. [Learn more](#)

Destination
Choose a destination to publish the event. [Learn more](#)

- ☒ **Lambda function**
Run a Lambda function script based on S3 events.
- ☐ **SNS topic**
Fanout messages to systems for parallel processing or directly to people.
- ☐ **SQS queue**
Send notifications to an SQS queue to be read by a server.

Specify Lambda function

- ☒ **Choose from your Lambda functions**
- ☐ Enter Lambda function ARN

Lambda function

Load-Inventory

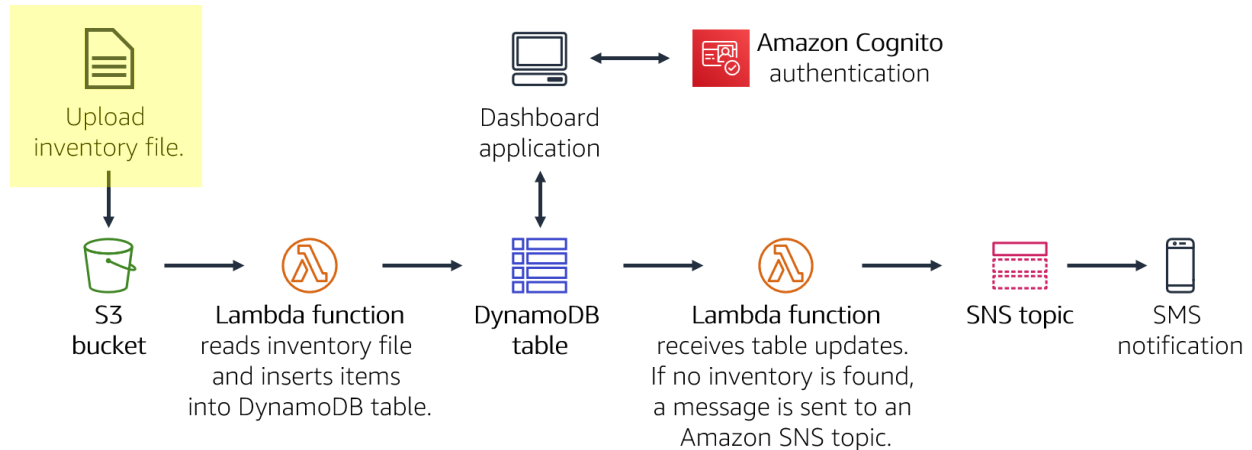
Cancel Save changes

CloudShell Feedback © 2023, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

21. When an object is created in the bucket, this configuration tells Amazon S3 to trigger the *Load-Inventory* Lambda function that you created earlier. Your bucket is now ready to receive inventory files!

Task 3: Testing the loading process

You are now ready to test the loading process. You will upload an inventory file, then check that it loaded successfully.



21. Download the inventory files by opening the context (right-click) menu for these links:

[inventory-berlin.csv](#)
[inventory-calcutta.csv](#)
[inventory-karachi.csv](#)
[inventory-pusan.csv](#)
[inventory-shanghai.csv](#)
[inventory-springfield.csv](#)

These files are the inventory files that you can use to test the system. They are comma-separated values (CSV) files. The following example shows the contents of the Berlin file:

```
store,item,count
Berlin,Echo Dot,12
Berlin,Echo (2nd Gen),19
Berlin,Echo Show,18
Berlin,Echo Plus,0
Berlin,Echo Look,10
Berlin,Amazon Tap,15
```

22. In the console, return to your S3 bucket by choosing the **Objects** tab.

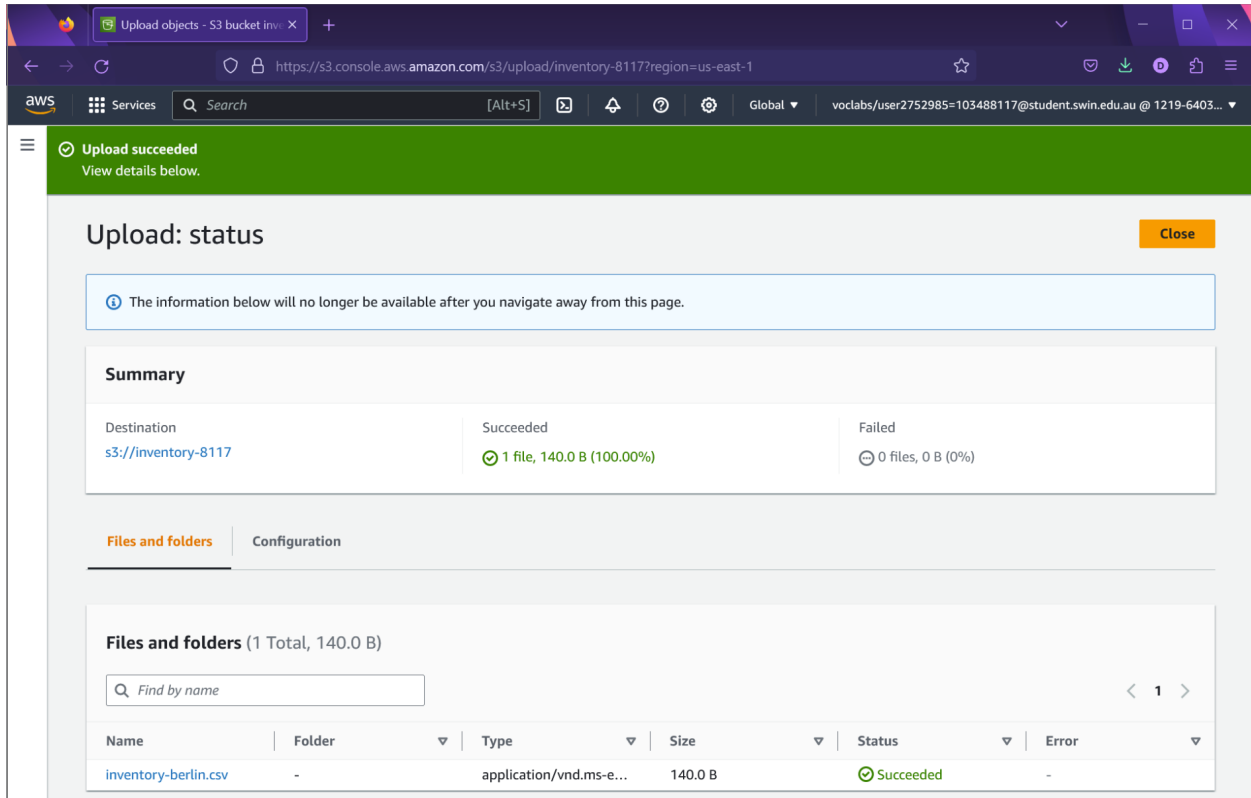
23. Choose Upload

24. Choose Add files, and select one of the inventory CSV files. (You can choose any inventory file.)

25. Choose Upload

Amazon S3 will automatically trigger the Lambda function, which will load the data into a DynamoDB table.

A serverless Dashboard application has been provided for you to view the results.

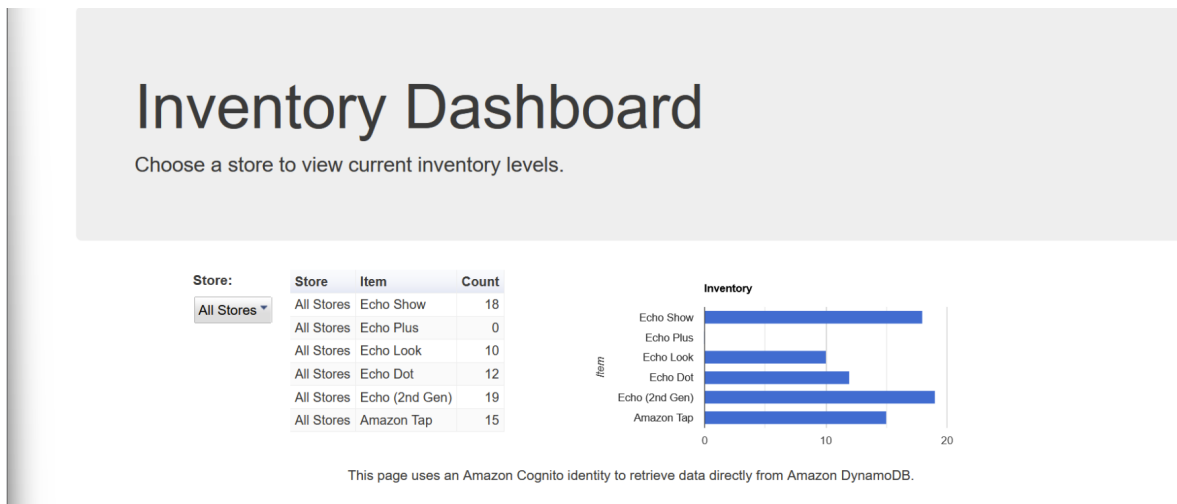


26. At the top of these instructions, choose the **Details** button, and to the right of **AWS**, choose the **Show** button.

27. From the **Credentials** window, copy the **Dashboard URL**.

28. Open a new web browser tab, paste the URL, and press ENTER.

The dashboard application will open and display the inventory data that you loaded into the bucket. The data is retrieved from DynamoDB, which proves that the upload successfully triggered the Lambda function.



Name: Trac Duc Anh Luong - ID: 103488117

If the dashboard application does not display any information, ask your instructor to help you diagnose the problem.

The dashboard application is served as a static webpage from Amazon S3. The dashboard authenticates via Amazon Cognito as an *anonymous user*, which provides sufficient permissions for the dashboard to retrieve data from DynamoDB.

You can also view the data directly in the DynamoDB table.

29. On the Services menu, choose **DynamoDB**.

30. In the left navigation pane, choose **Tables**.

31. Choose the **Inventory** table.

32. Choose the **Items** tab.

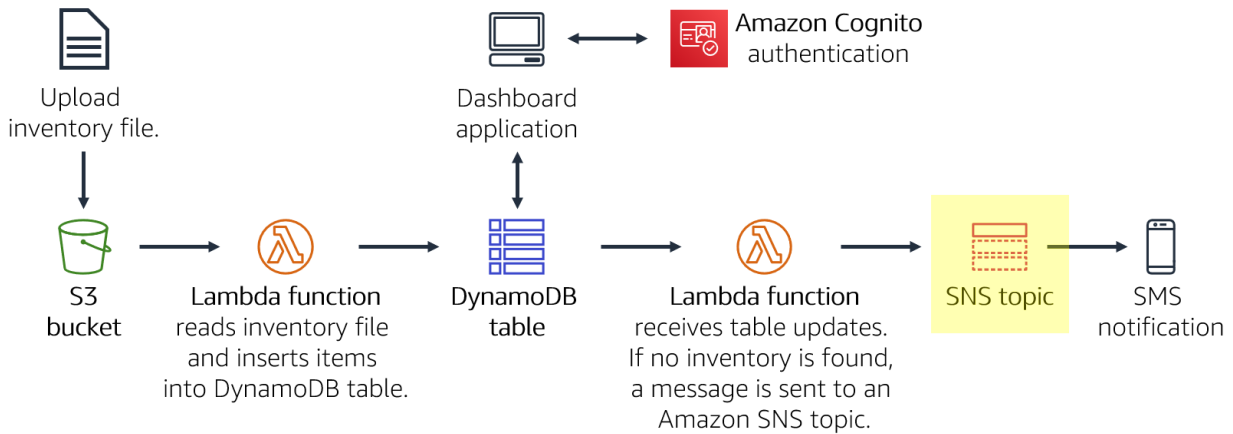
The data from the inventory file will be displayed. It shows the store, item and inventory count.

The screenshot shows the AWS Management Console interface for the DynamoDB service. The left navigation pane is open, showing the 'DynamoDB' section with options like 'Dashboard', 'Tables', 'Update settings', 'Explore items', 'PartiQL editor', 'Backups', 'Exports to S3', 'Imports from S3', 'Reserved capacity', and 'Settings'. The 'Explore items' option is selected. The main content area displays the 'Inventory' table. A 'Scan or query items' button is visible, along with a status message: 'Completed. Read capacity units consumed: 0.5'. Below this, a table titled 'Items returned (6)' shows the following data:

	Store (String)	Item (String)	Count
☐	Berlin	Amazon Tap	15
☐	Berlin	Echo (2nd Gen)	19
☐	Berlin	Echo Dot	12
☐	Berlin	Echo Look	10
☐	Berlin	Echo Plus	0
☐	Berlin	Echo Show	18

Task 4: Configuring notifications

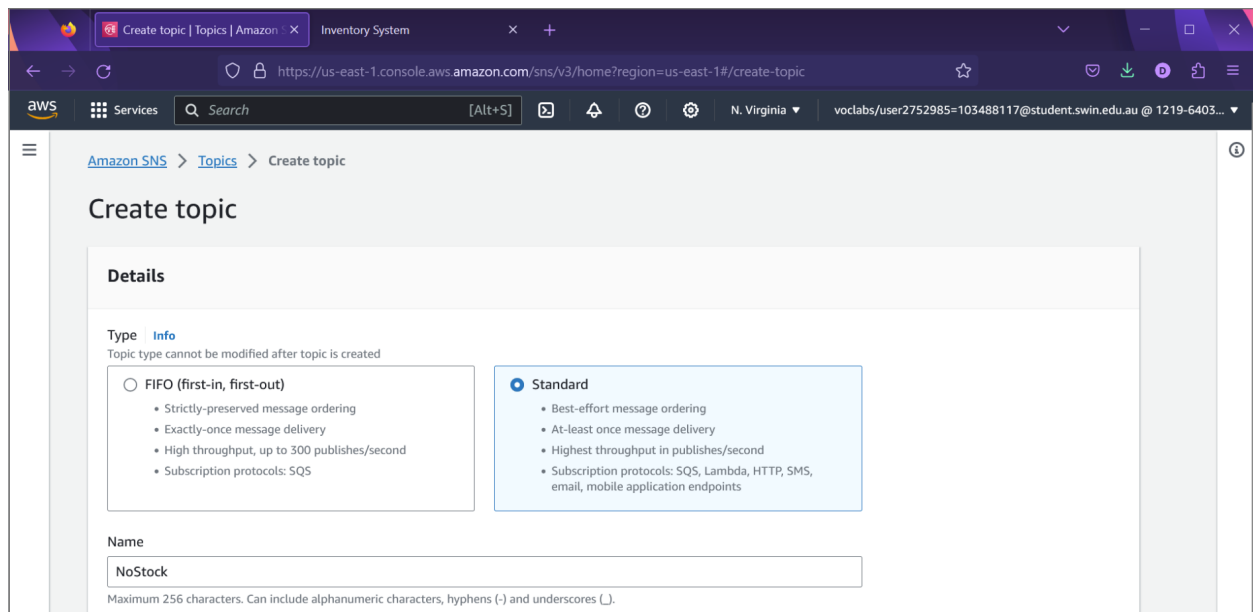
You want to notify inventory management staff when a store runs out of stock for an item. For this serverless notification functionality, you will use **Amazon SNS**.



Amazon SNS is a flexible, fully managed publish/subscribe messaging and mobile notifications service. It delivers messages to subscribing endpoints and clients. With Amazon SNS, you can fan out messages to a large number of subscribers, including distributed systems and services, and mobile devices.

33. On the Services menu, choose **Simple Notification Service**.

34. In the **Create topic** box, for **Topic name**, enter: **NoStock**. Keep **Standard** selected.



35. Choose Create topic

To receive notifications, you must *subscribe* to the topic. You can choose to receive notifications via several methods, such as SMS and email.

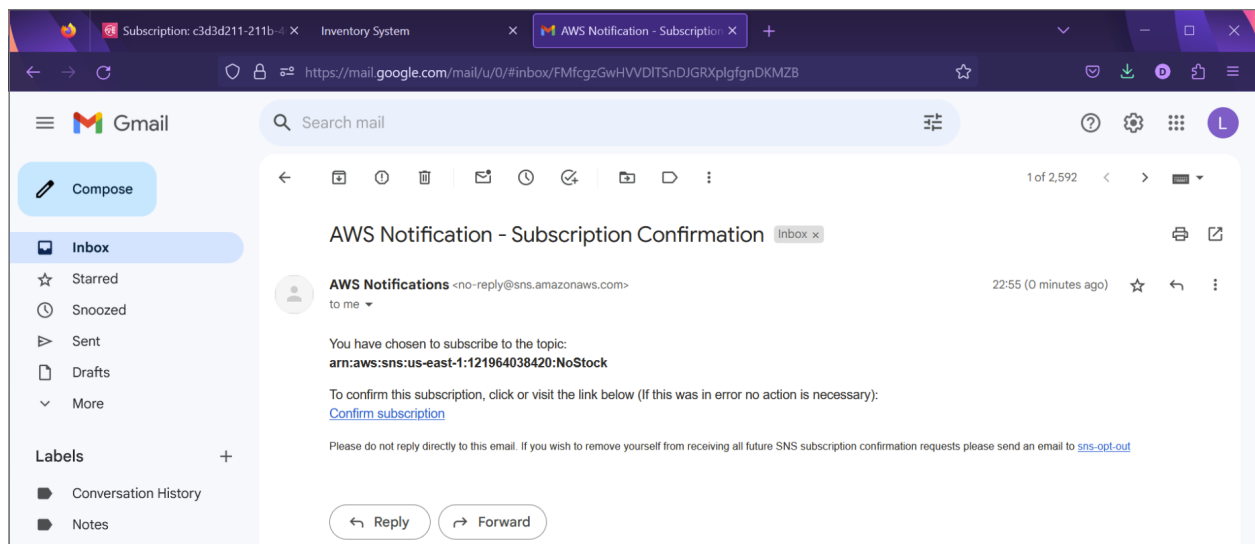
36. In the lower half of the page, choose Create subscription and configure these settings:

- **Protocol:** *Email*
- **Endpoint:** Enter your email address
- Choose Create subscription

The screenshot shows the AWS Management Console 'Create subscription' page. The browser address bar indicates the URL: <https://us-east-1.console.aws.amazon.com/sns/v3/home?region=us-east-1#/create-subscription>. The page title is 'Create subscription'. Under the 'Details' section, the 'Topic ARN' field contains 'arn:aws:sns:us-east-1:121964038420:NoStock'. The 'Protocol' dropdown menu is set to 'Email'. The 'Endpoint' field contains the email address 'luongtrac.ducanh@gmail.com'. A blue information box at the bottom states: 'After your subscription is created, you must confirm it. Info'.

37. After you create an email subscription, you will receive a confirmation email message. Open the message and choose the *Confirm subscription* link.

Any message that is sent to the SNS topic will be forwarded to your email.



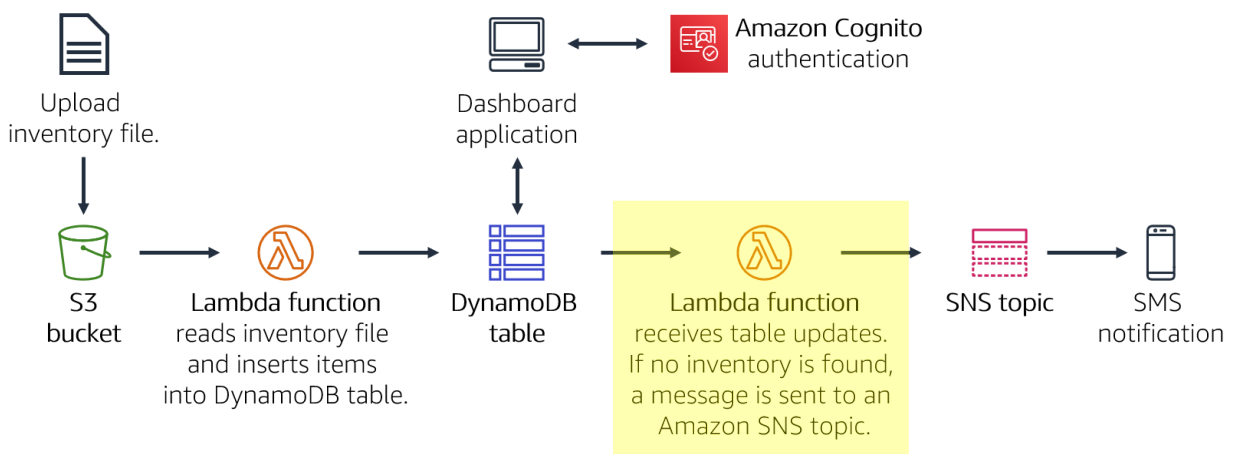
Task 5: Creating a Lambda function to send notifications

You could modify the existing *Load-Inventory* Lambda function to check inventory levels while the file is being loaded. However, this configuration is not a good architectural practice. Instead of overloading the *Load-Inventory* function with business logic, you will create another Lambda function that is triggered when data is loaded into the DynamoDB table. This function will be triggered by a *DynamoDB stream*.

This architectural approach offers several benefits:

- Each Lambda function performs a single, specific function. This practice makes the code simpler and more maintainable.
- Additional business logic can be added by creating additional Lambda functions. Each function operates independently, so existing functionality is not impacted.

In this task, you will create another Lambda function that looks at inventory while it is loaded into the DynamoDB table. If the Lambda function notices that an item is out of stock, it will send a notification through the SNS topic you created earlier.



39. On the Services menu, choose **Lambda**.

40. Choose Create function and configure these settings:

- **Function name:** **Check-Stock**
- **Runtime:** *Python 3.7*
- Expand **Choose or create an execution role**.
- **Execution role:** *Use an existing role*
- **Existing role:** *Lambda-Check-Stock-Role*
- Choose Create function

Name: Trac Duc Anh Luong - ID: 103488117

The screenshot shows the AWS Lambda 'Create function' console page. The browser tabs include 'Create function - Lambda', 'Inventory System', 'AWS Notification - Subscription', and 'Subscription confirm'. The URL is 'https://us-east-1.console.aws.amazon.com/lambda/home?region=us-east-1#/create/function'. The page has a dark header with the AWS logo and navigation links. The main content area is titled 'Create function' with an 'Info' link. Below the title, there's a message: 'AWS Serverless Application Repository applications have moved to [Create application](#).' There are three tabs: 'Author from scratch' (selected), 'Use a blueprint', and 'Container image'. The 'Basic information' section contains: 'Function name' (Check-Stock), 'Runtime' (Python 3.7), and 'Architecture' (x86_64). The 'Permissions' section shows 'Change default execution role' with options: 'Create a new role with basic Lambda permissions', 'Use an existing role' (selected), and 'Create a new role from AWS policy templates'. The 'Existing role' dropdown shows 'Lambda-Check-Stock-Role'. At the bottom, there are 'Cancel' and 'Create function' buttons.

This role was configured with permissions to send a notification to Amazon SNS.

41. Scroll down to the **Code source** section, and in the **Environment** pane, choose `lambda_function.py`.
42. In the code editor, delete all the code.
43. Copy the following code, and in the **Code Source** editor, paste the copied code:

```
# Stock Check Lambda function
#
# This function is triggered when values are inserted into the Inventory DynamoDB table.
# Inventory counts are checked and if an item is out of stock, a notification is sent to an SNS
Topic.
import json, boto3
# This handler is run every time the Lambda function is triggered
def lambda_handler(event, context):
    # Show the incoming event in the debug log
    print("Event received by Lambda function: " + json.dumps(event, indent=2))
    # For each inventory item added, check if the count is zero
    for record in event['Records']:
        newImage = record['dynamodb'].get('NewImage', None)
        if newImage:
            count = int(record['dynamodb']['NewImage']['Count']['N'])
            if count == 0:
                store = record['dynamodb']['NewImage']['Store']['S']
                item = record['dynamodb']['NewImage']['Item']['S']
                # Construct message to be sent
                message = store + ' is out of stock of ' + item
                print(message)
                # Connect to SNS
                sns = boto3.client('sns')
                alertTopic = 'NoStock'
                snsTopicArn = [t['TopicArn'] for t in sns.list_topics()['Topics']
                               if t['TopicArn'].lower().endswith(':' + alertTopic.lower())][0]
                # Send message to SNS
                sns.publish(
                    TopicArn=snsTopicArn,
                    Message=message,
                    Subject='Inventory Alert!',
                    MessageStructure='raw'
                )
    # Finished!
    return 'Successfully processed {} records.'.format(len(event['Records']))
```

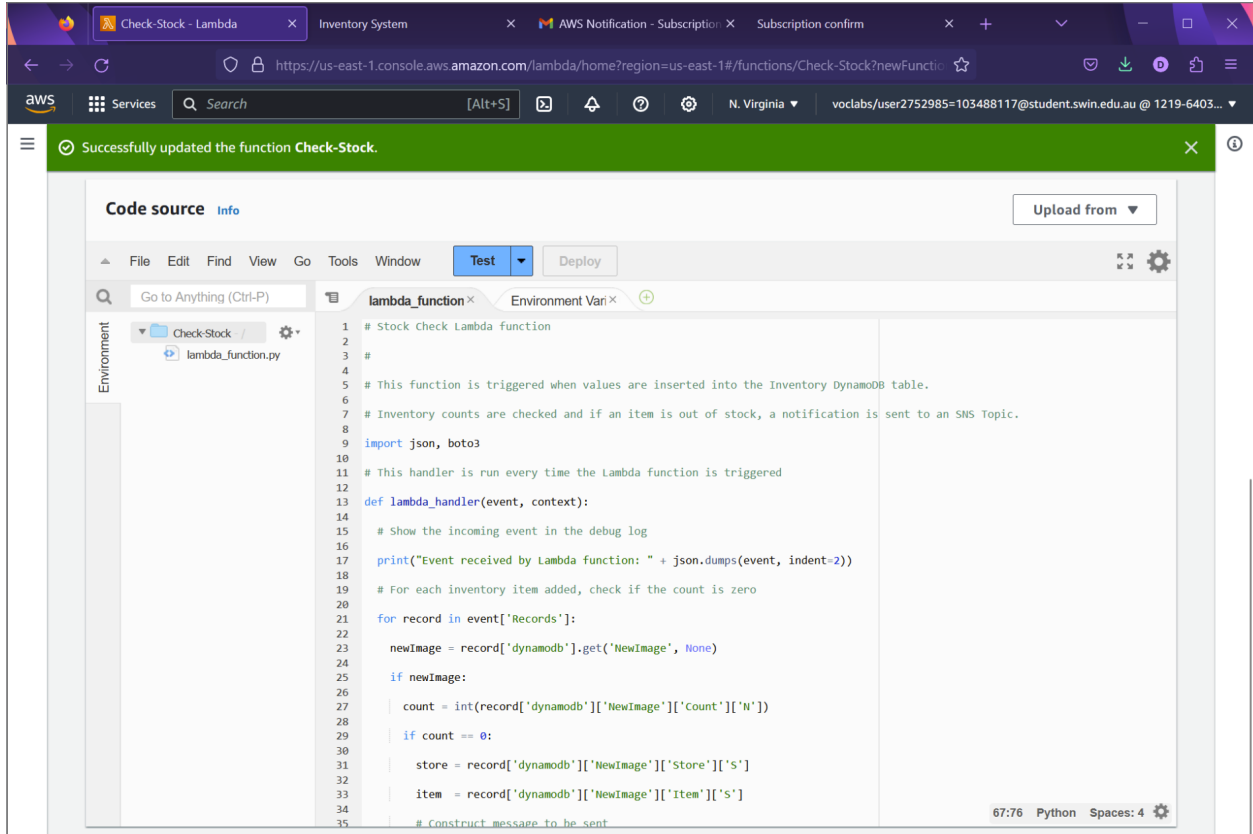
Examine the code. It performs the following steps:

- Loop through the incoming records
- If the inventory count is zero, send a message to the *NoStock* SNS topic

You will now configure the function so it triggers when data is added to the *Inventory* table in DynamoDB.

44. Choose Deploy to save your code changes

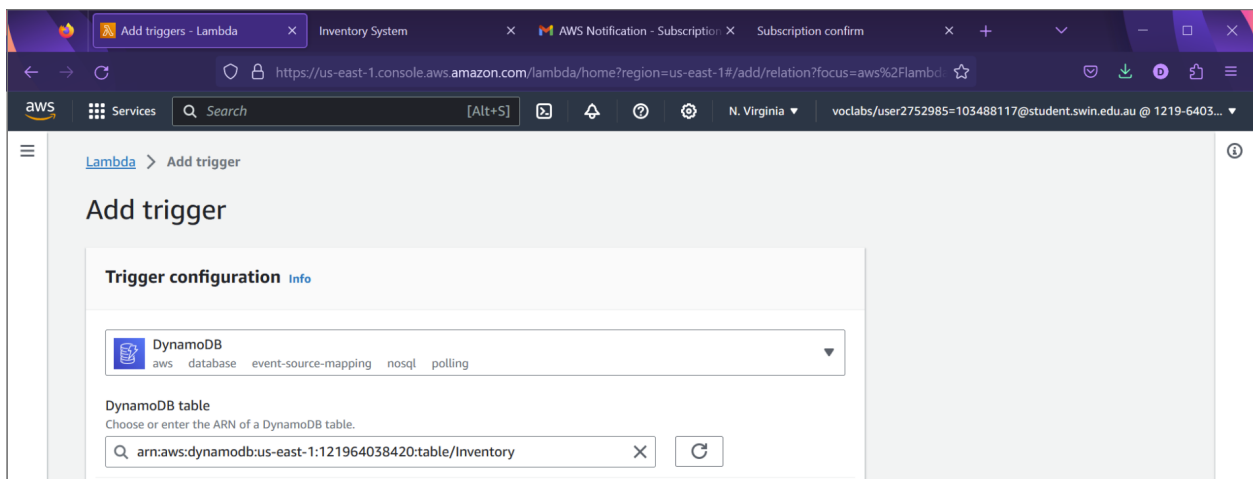
Name: Trac Duc Anh Luong - ID: 103488117



45. Scroll to the **Designer** section (which is at the top of the page).

46. Choose Add trigger and then configure these settings:

- **Select a trigger:** *DynamoDB*
- **DynamoDB Table:** *Inventory*
- Choose Add



You are now ready to test the system!

Task 6: Testing the System

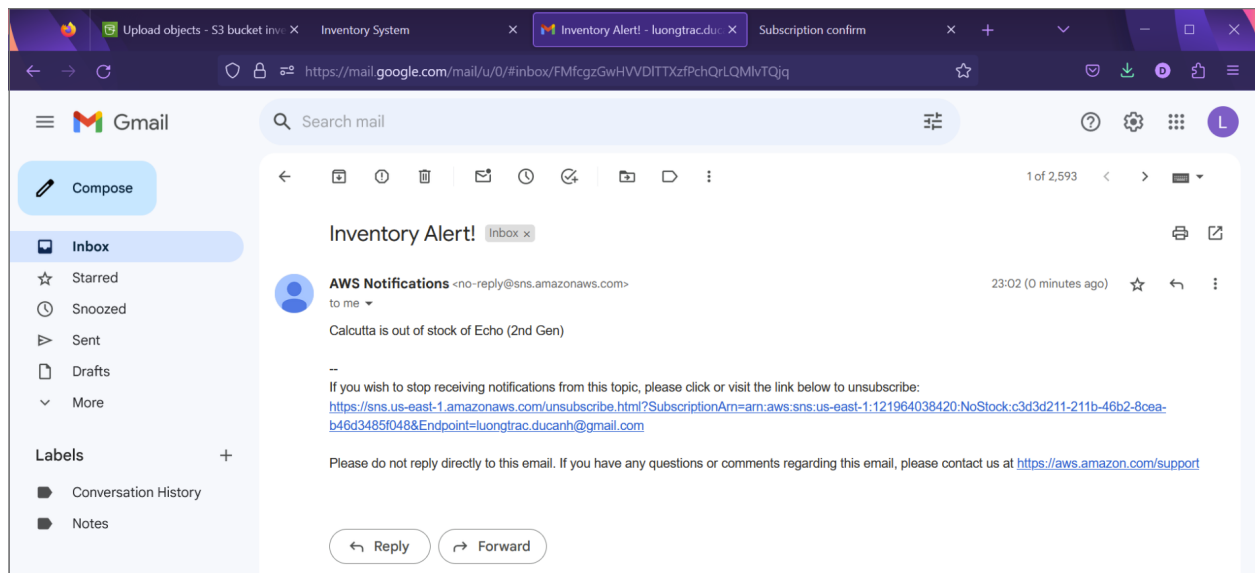
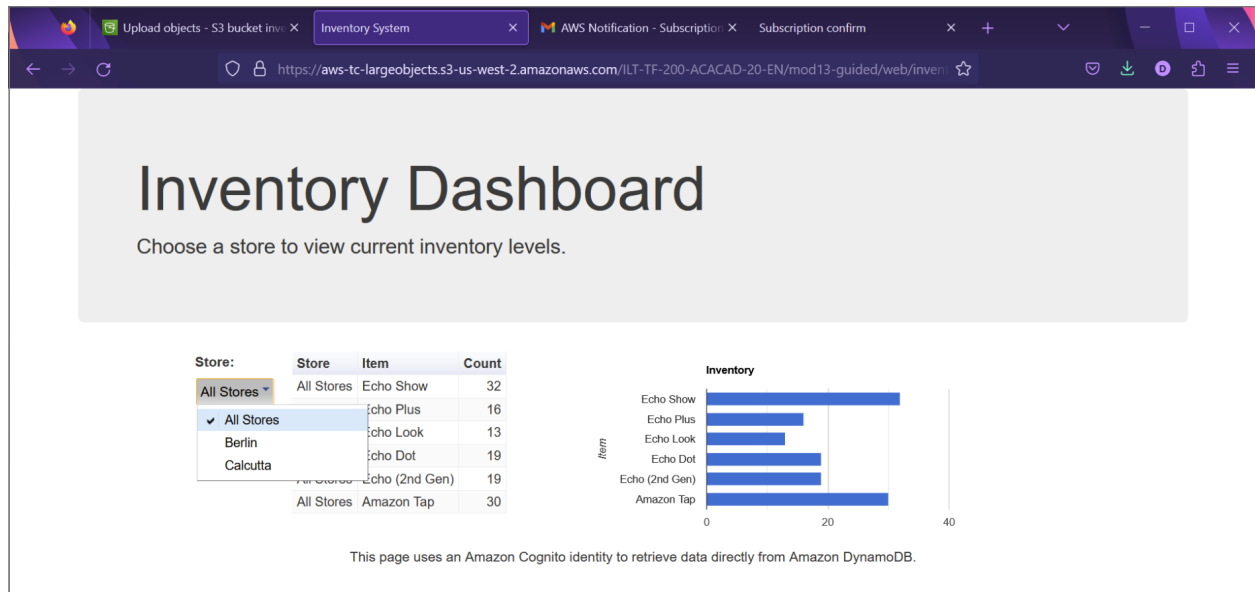
You will now upload an inventory file to Amazon S3, which will trigger the original *Load-Inventory* function. This function will load data into DynamoDB, which will then trigger the new *Check-Stock* Lambda function. If the Lambda function detects an item with zero inventory, it will send a message to Amazon SNS. Then, Amazon SNS will notify you through SMS or email.

47. On the Services menu, choose **S3**.
48. Choose the name of your *inventory*- bucket.
49. Choose **Upload** and upload a different inventory file.

The screenshot shows the AWS S3 console interface. At the top, a green banner indicates "Upload succeeded" with a link to "View details below." Below this, the "Upload: status" section is displayed. A summary box shows the upload details: Destination is "s3://inventory-8117", 1 file (150.0 B) was successfully uploaded (100.00%), and 0 files (0 B) failed (0%). Below the summary, the "Files and folders" tab is selected, showing a table with 1 total file. The table has columns for Name, Folder, Type, Size, Status, and Error. The file "inventory-calcutta.csv" is listed with a status of "Succeeded".

Name	Folder	Type	Size	Status	Error
inventory-calcutta.csv	-	application/vnd.ms-e...	150.0 B	Succeeded	-

50. Return to the **Inventory System Dashboard** and refresh the page.
You should now be able to use the **Store** menu to view the inventory from both stores.
Also, you should receive a *notification through SMS or email* that the store has an out-of-stock item (each inventory file has one item that is out of stock).
If you did not receive a notification, wait a few minutes and upload a different inventory file. The DynamoDB trigger can sometimes take a few minutes to enable.



51. Try to upload multiple inventory files at the same time. What do you think will happen?

Uploading multiple inventory files simultaneously to the S3 bucket will trigger the Lambda function separately for each file that gets uploaded. Each upload event will independently invoke the Lambda function "Load-Inventory" for processing the inventory data.

Each file uploaded will trigger the Lambda function to download, process, and insert the data into the DynamoDB table separately. This means that each file will be processed in its instance of the Lambda function execution without direct influence or interference between the uploaded files.

Submitting your work

52. At the top of these instructions, choose Submit to record your progress and when prompted, choose **Yes**.
53. If the results don't display after a couple of minutes, return to the top of these instructions and choose Grades
Tip: You can submit your work multiple times. After you change your work, choose **Submit** again. Your last submission is what will be recorded for this lab.
54. To find detailed feedback on your work, choose Details followed by **View Submission Report**.

The screenshot shows the AWS Academy Instructure interface for a guided lab. The page title is "Module 13 Guided Lab - Implementing a Serverless Architecture with AWS Lambda". The submission status is "Submitting an external tool". The submission report shows a total score of 40/40 and a list of tasks completed:

- [Task 1A] Load-Inventory Function Crea
- [Task 1B] Load-Inventory Execution Rol
- [Task 2A] Bucket Created
- [Task 2B] Bucket Event
- [Task 3] Database Table Data
- [Task 4] SNS Topic Created
- [Task 5A] Check-Stock Function Created
- [Task 5B] Check-Stock has Trigger

The submission report also includes a "Submission Report" section with the following details:

```
[Executed at: Tue Nov 7 8:05:18 PST 2023]

Testing report - The function named Load-Inventory was found.
Testing report - The correct Role was used for the Load-Inventory function.
Testing report - Bucket named,inventory-8117,found.
Testing report - Bucket,inventory-8117,was cofigured with a Lambda event.
Testing report - Records were inserted into the Inventory DynamoDB table.
Testing report - The topic, NoStock, was created.
Testing report - The fuction Check-Stock was found.
Testing report - A trigger was found for the Check-Stock lambda funtion.

gradeFile = /mnt/vocwork2/ccc_v1_g_11ed7_28593/asn2061457_17/asn2061458_1/tm
reportFile = /mnt/vocwork2/ccc_v1_g_11ed7_28593/asn2061457_17/asn2061458_1/tm
/mnt/vocwork2/ccc_v1_g_11ed7_28593/asn2061457_17/tmp/temp_uf_11
len 4
Present working directory = /mnt/vocwork2/ccc_v1_g_11ed7_28593/asn2061457_17
```